

SIMATS ENGINEERING

Assessment - CO2-AT2

Subject: Artificial Intelligence

Course Code: CSA 1712

Name: Vignesh Kumar K

Register No: 192512375

Course Outcome Covered (CO2)

Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Question 1: AI-Powered Drones for Emergency Medicine Delivery

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic — roads may suddenly become inaccessible, and weather affects travel cost. The drone has a heuristic estimate (straight-line distance), partial real-time updates on blocked paths, and variable movement costs. Decisions must minimise delay and risk.

i. Behaviour of Greedy Best-First Search (GBFS)

GBFS selects the next node to expand purely on the heuristic value $h(n)$ — the straight-line distance to the destination — while ignoring the actual cost $g(n)$ already incurred. In this scenario, the drone would always head toward the neighbouring point that looks closest to the destination in a straight line, regardless of terrain difficulty or weather-related cost.

Strengths: Extremely fast decision-making with low memory and computational overhead, which suits the drone's need for quick real-time responses during an emergency; it requires no bookkeeping of cumulative cost.

Weaknesses: It is neither optimal nor complete — it can be misled by a locally attractive heuristic into flying toward a blocked or dangerous route, since it never reconsiders the true cost of the path taken. Under uncertainty (sudden blockages, changing weather), it may commit to a path that looks short on the map but is actually slow, unsafe, or impassable.

ii. Behaviour of A* Search

A* evaluates every candidate node using $f(n) = g(n) + h(n)$, combining the actual cost travelled so far (g) with the estimated remaining cost (h). For the drone, this means it balances the real cost of terrain and weather already encountered against the straight-line estimate of the remaining distance, so it does not blindly chase the geometrically shortest-looking route. If the heuristic is admissible (never overestimates), A* is guaranteed to find the lowest-cost, safest route rather than merely the most direct-looking one, which is critical when variable terrain and weather costs are involved.

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search, heuristic accuracy is everything — because cost is ignored entirely, an inaccurate heuristic (e.g. straight-line distance that ignores flood-blocked roads) directly leads to poor, unsafe, or failed routes. For A*, an admissible and consistent heuristic guarantees the optimal path; if the heuristic underestimates cost (as straight-line distance typically does), A* still finds the optimal route but explores more nodes, so it becomes slower; only an inadmissible heuristic (overestimating cost) would break A*'s optimality guarantee. Overall, A* is far more robust to heuristic imprecision than Greedy search, whose correctness collapses with a poor heuristic.

iv. Effect of Dynamic Changes (Blocked Paths)

Both algorithms are designed around a graph that is assumed static during search; when a path becomes blocked mid-flight, neither will automatically notice unless the search is re-run. Greedy search is more fragile here because it commits early to whichever direction appears closest, so a late-discovered blockage forces a costly full restart with no memory of cost already spent. A* retains the accumulated cost information, which

makes replanning comparatively less wasteful, and it is the natural basis for dynamic/real-time variants such as D* Lite or Anytime Repairing A*, which are built specifically to incrementally repair a plan as new blockage information arrives rather than restarting from scratch.

v. Recommendation

A* — ideally an incremental, real-time variant such as D* Lite or Anytime A* — is the most suitable choice for this scenario. It balances actual travel cost (accounting for terrain and weather) with heuristic guidance (straight-line distance) to produce routes that are both fast to compute and close to optimal, while its cost-aware structure supports efficient replanning as blocked paths are reported. Pure Greedy Best-First Search, while faster to compute, is too easily misled under uncertainty and cannot guarantee the safety and optimality required when delivering emergency medicine, so it is not recommended as the primary strategy.

Question 2: Smart City Traffic Signal Optimisation

Scenario: An AI system must optimise traffic signal timings across hundreds of intersections to minimise total waiting time, fuel consumption, and congestion, while continuously adapting to changing traffic flow in an extremely large search space.

Problem Formulation and Why Traditional Search is Inefficient

This is best formulated as an optimisation problem rather than a classic path-finding problem: the state is a complete configuration of signal timings across all intersections, and the objective function is a weighted combination of total waiting time, fuel consumption, and congestion level, to be minimised. There is no single fixed start/goal path to search for — instead the system must continually find the best configuration among an astronomically large combinatorial space of possible timings. Traditional systematic search algorithms (BFS, DFS, A*) are inefficient here because the state space grows exponentially with the number of intersections, exhaustive exploration cannot possibly finish before traffic conditions change again, and there is no natural notion of a 'goal state' to search toward — the problem calls for local/optimisation search rather than path search.

i. Hill Climbing and Its Limitations

Hill climbing starts from an initial timing configuration and repeatedly moves to a neighbouring configuration (a small change in one or more signal timings) if it improves the objective function, stopping when no neighbour is better. Its major limitation is short-sightedness: it only ever looks at immediate neighbours, has no mechanism to backtrack, and therefore can settle on a configuration that is only locally good rather than city-wide optimal.

ii. Local Maxima, Plateaus, and Ridges

Local maxima: a signal-timing pattern where every small adjustment makes waiting time/congestion worse, even though a fundamentally different combination of timings elsewhere in the search space would perform far better city-wide — e.g. a cluster of intersections looks well-tuned locally while overall city traffic remains congested.

Plateaus: regions where changing timings produces no measurable improvement (e.g. traffic is already evenly distributed across nearby signal phases), leaving hill climbing with no gradient to follow.

Ridges: narrow paths of improvement that require simultaneous, coordinated changes across multiple intersections (e.g. a 'green wave' along a corridor); single-step hill climbing that changes one signal at a time cannot follow this diagonal path efficiently.

iii. Overcoming These Limitations

Simulated Annealing: borrows the metallurgical annealing idea of occasionally accepting a worse configuration with a probability that decreases over time (a cooling schedule). This lets the search escape local maxima and cross plateaus early on, while gradually converging to a strong solution as the 'temperature' drops.

Genetic Algorithms: maintain a population of candidate timing configurations and evolve them using selection, crossover, and mutation across generations. Because many regions of the solution space are explored in parallel, the risk of being trapped in one local optimum is greatly reduced, and the population can keep adapting as traffic patterns change and fitness is re-evaluated.

iv. Recommendation

Given the scale (hundreds of intersections) and the need for continuous adaptation to changing traffic, a Genetic Algorithm — ideally hybridised with a local-search refinement step such as Simulated Annealing (a memetic approach) — is the most suitable. Its population-based, parallel search scales better to very large combinatorial spaces and naturally re-adapts as fitness values shift with real-time traffic, whereas pure Simulated Annealing, though simpler to tune, adapts more slowly at this scale.

Question 3: Autonomous Mars Rover Exploration

Scenario: A rover must navigate unknown Martian terrain to collect samples, avoid hazards, and make decisions in real time with limited sensing and no complete map, updating its knowledge as it explores.

i. Why an Online Search Agent Is Required

Offline search assumes the agent has complete knowledge of the environment before it starts moving, so it can compute an entire solution path in advance. The rover does not have this luxury — the terrain map is unknown at the outset — so it must interleave computation with action: take a step, sense the result, update its knowledge, and only then decide the next action. This act-observe-update cycle is the defining characteristic of an online search agent.

ii. Challenges of Partial Observability and Dynamic Environments

Because the rover's sensors only cover a limited local range, it must act on incomplete information about what lies beyond that range, risking encounters with hazards (craters, rocks) it could not have foreseen. This is compounded by the possibility of the environment itself changing (dust movement, minor rockslides), and by the significant communication delay to Earth, which rules out relying on real-time remote assistance — the rover must be able to safely handle unexpected situations autonomously and conservatively.

iii. Building and Updating the Internal Model

As it explores, the rover records what it has directly observed — free cells, obstacles, hazard locations — into an internal map (a process akin to simultaneous localisation and mapping). Each new observation updates this partial map: cells found to be blocked are marked as impassable, previously assumed-safe routes are corrected if hazards are discovered, and this continually revised model is what the rover uses to plan its next moves, rather than any pre-supplied full map.

iv. Online Search vs. Uninformed and Informed Search

Uninformed search (e.g. BFS, DFS) uses no domain knowledge, exploring blindly; it is complete but far too inefficient for a resource- and time-constrained rover. Informed (offline) search uses heuristics for efficient planning but presumes a fully known map, which the rover does not have. Online search differs from both because it must choose actions using only currently known information, cannot perform full lookahead into unexplored territory, and gains information incrementally through action — algorithms such as Learning Real-Time A* (LRTA*) combine heuristic guidance (like informed search) with the incremental, action-then-observe structure required for genuinely unknown environments.

v. Justification of the Most Suitable Approach

An online, heuristic-guided search approach such as LRTA* is the most suitable choice. It lets the rover use heuristic estimates to move efficiently toward its goal while continuously updating its internal map and revising decisions as hazards are discovered, which directly addresses the rover's core constraints of partial observability, a changing environment, and the safety-critical need to avoid committing to a plan based on stale or incomplete information.

Question 4: University Exam Timetable Generation (CSP)

Scenario: A university must automatically generate exam timetables that avoid overlapping exams for any student, respect limited exam hall capacity, honour subject ordering, faculty availability, and special accommodations — with complexity growing as courses and students increase.

Formulating the Problem as a CSP

i. Variables, Domains, and Constraints

Variables: one variable per exam/subject to be scheduled, e.g. $X_1, X_2, \dots, X_n$, each representing a subject that needs a timetable slot.

Domains: for each variable, the domain is the set of feasible (timeslot, exam hall) pairs available for that subject, given the exam period's overall slots and halls.

Constraints: (a) no-overlap constraint — any two exams sharing at least one common student must be assigned different timeslots; (b) capacity constraint — the number of exams scheduled in the same timeslot cannot exceed the number of available halls; (c) precedence constraint — certain subjects must be scheduled strictly before others (e.g. prerequisite before advanced subject); (d) faculty-availability constraint — an exam's timeslot must fall within a time when its invigilating faculty is available; (e) accommodation constraint — students needing extended time or a separate hall require dedicated slot/hall assignments that do not conflict with their other exams.

ii. How Backtracking Search Works Here

Backtracking search assigns a (timeslot, hall) value to one exam variable at a time, checking after each assignment whether all constraints involving already-assigned variables are still satisfied. If an assignment violates a constraint (e.g. two exams with common students land on the same slot), the search backtracks — undoes the most recent assignment(s) — and tries the next available value, continuing until either a fully consistent timetable is found or the search space is exhausted.

iii. How Constraint Propagation (Forward Checking) Improves Efficiency

Forward checking looks ahead each time a variable is assigned: it removes, from the domains of unassigned variables directly constrained with the one just assigned, any value that would immediately violate a constraint. For example, once one subject is fixed to a slot/hall, that slot is removed from the domain of any exam sharing a student with it, and the hall's remaining capacity is updated for every other exam considering that slot. This detects dead ends far earlier than plain backtracking, considerably reducing the number of wasted assignments and backtracks, and can be strengthened further with full arc-consistency techniques such as AC-3.

iv. Real-World Challenges and the Best Strategy for Scalability

As the number of courses and students grows, the constraint graph becomes dense, and worst-case backtracking search can become exponentially slow; conflicting constraints (e.g. limited faculty availability clashing with accommodation needs) can also create regions of the search space with no feasible solution, requiring constraint relaxation or prioritisation. For real-world scalability, backtracking search should be combined with variable-and-value ordering heuristics — Minimum Remaining Values (choosing the most constrained exam next) and Least Constraining Value (choosing the slot/hall that rules out the fewest options for others) — together with forward checking or arc-consistency for propagation, and local-repair techniques such as min-conflicts for very large instances. This combination offers the best balance of completeness and practical efficiency for large-scale university timetabling.

Question 5: Strategic Game AI (Minimax & Alpha-Beta Pruning)

Scenario: A gaming company is building a real-time strategy AI opponent that must compete with human players, make decisions within strict time limits, and evaluate an extremely large decision tree of possible moves.

i. How Minimax Models the Problem

Minimax represents the game as a tree of alternating states: the AI's turn (maximiser), trying to pick the move that leads to the best outcome for itself, and the opponent's turn (minimiser), assumed to always pick the move that is worst for the AI. Values from terminal (or cut-off) states are scored and propagated back up the tree — maximiser nodes take the highest child value, minimiser nodes take the lowest — so the AI's chosen move at the root reflects the best outcome it can guarantee against a rational, worst-case-playing opponent.

ii. Computational Challenges of Large Game Trees

Real-time strategy games have a very high branching factor (many simultaneous unit/move options) and can require significant search depth to plan meaningfully ahead, so the number of nodes to evaluate grows exponentially as $O(b^d)$, where b is the branching factor and d is the search depth. Within the strict time limits

of real-time play, exploring the full tree to any useful depth becomes computationally infeasible, and the memory needed to hold the expanding tree also grows rapidly.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-beta pruning tracks two bounds while traversing the tree: alpha, the best value the maximiser can already guarantee, and beta, the best value the minimiser can already guarantee. Whenever a node's value is found to be no better than a bound already secured elsewhere in the tree ($\alpha \geq \beta$), the remaining branches under that node cannot possibly change the final decision and are pruned without being explored. This does not change the final chosen move — it is mathematically equivalent to full minimax — but with good move ordering it can reduce the effective time complexity from $O(b^d)$ toward $O(b^{(d/2)})$, allowing roughly twice the search depth within the same time budget.

iv. Designing an Evaluation Function

Since the full tree cannot be searched to terminal states within real-time limits, non-terminal states must be scored using a heuristic evaluation function that estimates how favourable a position is. A reasonable evaluation function would combine several weighted factors: material/unit and resource count difference, territory or map control, health/damage differential between forces, positional advantage (e.g. proximity to key objectives), and economic/tempo strength (resource production rate). The weight given to each factor should reflect its real influence on winning, tuned through testing or learning so the function's output correlates well with actual win likelihood.

v. Strategy for Balancing Optimality and Real-Time Performance

The recommended strategy combines alpha-beta pruning with iterative deepening — searching progressively deeper levels and always keeping the best move found so far, so a reasonable move is available the instant the time budget expires — together with good move-ordering heuristics to maximise pruning effectiveness, and transposition tables to avoid re-evaluating previously seen states. Together these techniques let the AI make near-optimal decisions while reliably respecting the strict time constraints of real-time strategy gameplay.